გაკვეთილი 7 Mod 7 (7.2)

გაკვეთილი 3 — GitHub-ზე Branch-ების ატვირთვა და გაერთიანება (Push, Merge, Merge Conflict)

2. თეორიული ნაწილი

2.1. წინა გაკვეთილების გამეორება

წინა გაკვეთილებზე ვისწავლეთ, როგორ შევქმნათ Git რეპოზიტორია და როგორ ვიმუშაოთ Branch-ებთან.

მაგალითად, გვაქვს ვებსაიტის პროექტი:

```
school-website/
    index.html
    style.css
    script.js
```

შევქმენით სამი Branch:

```
main
header
footer
```

თითოეულ Branch-ში განსხვავებული ცვლილებები შევიტანეთ.

მაგალითად:

- `main` — ვებსაიტის ძირითადი ვერსია;
- `header` — დამატებული აქვს ნავიგაციის მენიუ;
- `footer` — დამატებული აქვს გვერდის ქვედა ნაწილი.

მაგრამ აქამდე ეს ცვლილებები მხოლოდ ჩვენს კომპიუტერში ინახებოდა.

როგორ გავუზიაროთ ჩვენი ცვლილებები გუნდის სხვა წევრებს?

ამისთვის საჭიროა ცვლილებების GitHub-ზე ატვირთვა.

2.2. რა არის Git Push?

`git push` არის Git-ის ბრძანება, რომელიც ლოკალურ რეპოზიტორიაში შექმნილ commit-ებს დისტანციურ რეპოზიტორიაში აგზავნის.

მაგალითად, ჩვენს კომპიუტერში გვაქვს:

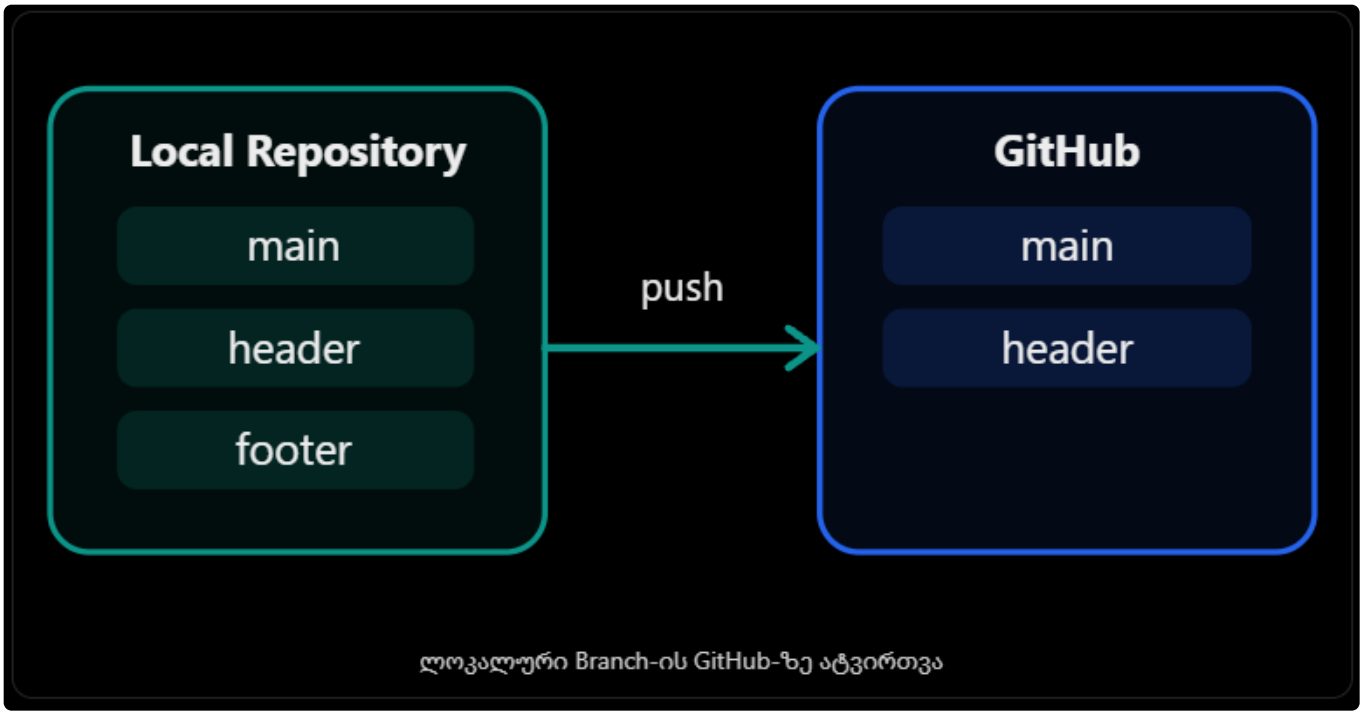
```
Local Repository
    main
    header
    footer
```

ხოლო GitHub-ზე თავდაპირველად შეიძლება მხოლოდ `main` Branch არსებობდეს.

თუ გვინდა `header` Branch-ის ატვირთვა, გამოვიყენებთ:

```
git push -u origin header
```

ამ ბრძანების შესრულების შემდეგ GitHub-ზე გაჩნდება `header` Branch და მასში არსებული commit-ები.



ლოკალური Branch-ის GitHub-ზე ატვირთვა

ლოკალური Branch-ის GitHub-ზე ატვირთვა

მნიშვნელოვანია: Git-ის `commit` და `push` სხვადასხვა მოქმედებებია.

ბრძანება	დანიშნულება
`git add`	ცვლილებების მომზადება commit-ისთვის
`git commit`	ცვლილებების ლოკალურ ისტორიაში შენახვა

ბრძანება	დანიშნულება
`git push`	commit-ების დისტანციურ რეპოზიტორიაში ატვირთვა

მაგალითად:

```
git add .
git commit -m "Added navigation menu"
git push
```

ბოლო ბრძანება იმუშავებს, თუ მიმდინარე Branch-ს უკვე გამართული დისტანციური კავშირი (upstream) აქვს. წინააღმდეგ შემთხვევაში შესაბამისი Branch და კავშირი ჯერ უნდა განვსაზღვროთ.

2.3. Local Branch და Remote Branch

Local Branch არის Branch, რომელიც ჩვენს კომპიუტერში არსებობს.

Remote Branch არის დისტანციურ რეპოზიტორიაში არსებული Branch, მაგალითად GitHub-ზე.

შეგვიძლია შევამოწმოთ ლოკალური Branch-ები:

```
git branch
```

მაგალითად:

```
* header
  main
  footer
```

ხოლო Git-ისთვის ცნობილი დისტანციური Branch-ების სანახავად:

```
git branch -r
```

მაგალითად:

```
origin/main
origin/header
```

`origin` არის დისტანციური რეპოზიტორიის სახელი, რომელიც ჩვეულებრივ ავტომატურად იქმნება `git clone` ბრძანების დროს.

`origin/header` ნიშნავს დისტანციური `header` Branch-ის ლოკალურად ცნობილ მდგომარეობას. GitHub-ზე უშუალოდ მიმდინარე მდგომარეობის გასაგებად შეიძლება საჭირო იყოს `git fetch`.

2.4. როგორ ავტვირთოთ Branch GitHub-ზე?

განვიხილოთ მაგალითი.

ვქმნით ახალ Branch-ს:

```
git checkout -b navbar
```

`index.html`-ში ვამატებთ ნავიგაციის მენიუს:

```
<nav>
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
    <a href="contact.html">Contact</a>
</nav>
```

შემდეგ ვინახავთ:

```
git add .
git commit -m "Added navbar"
```

ახლა საჭიროა Branch-ის GitHub-ზე ატვირთვა:

```
git push -u origin navbar
```

განვიხილოთ ბრძანება ნაწილებად:

- `git push` — ცვლილებების გაგზავნა;
- `-u` — upstream კავშირის დამახსოვრება;
- `origin` — დისტანციური რეპოზიტორია;
- `navbar` — Branch, რომლის ატვირთვაც გვინდა.

ამის შემდეგ, ჩვეულებრივ, საკმარისი იქნება:

```
git push
```

რადგან Git-ს უკვე ეცოდინება, რომელ დისტანციურ Branch-თან არის დაკავშირებული `navbar`.

ყურადღება: `git push` არ ქმნის commit-ს. ის აგზავნის უკვე შექმნილ commit-ებს. ატვირთვისთვის საჭიროა დისტანციურ რეპოზიტორიაზე ჩაწერის უფლება.

2.5. რა არის Git Merge?

წარმოიდგინეთ, რომ ვებსაიტზე სხვადასხვა ფუნქციას ვქმნით.

`main` Branch-ში გვაქვს ძირითადი გვერდი.

`navbar` Branch-ში შევქმენით ნავიგაციის მენიუ.

ახლა გვინდა, რომ ნავიგაციის მენიუ მთავარ პროექტშიც გამოჩნდეს.

ამისთვის ვიყენებთ `git merge` ბრძანებას.

`git merge` არის ბრძანება, რომელიც ერთი Branch-ის ცვლილებებს სხვა Branch-ის ისტორიაში აერთიანებს.

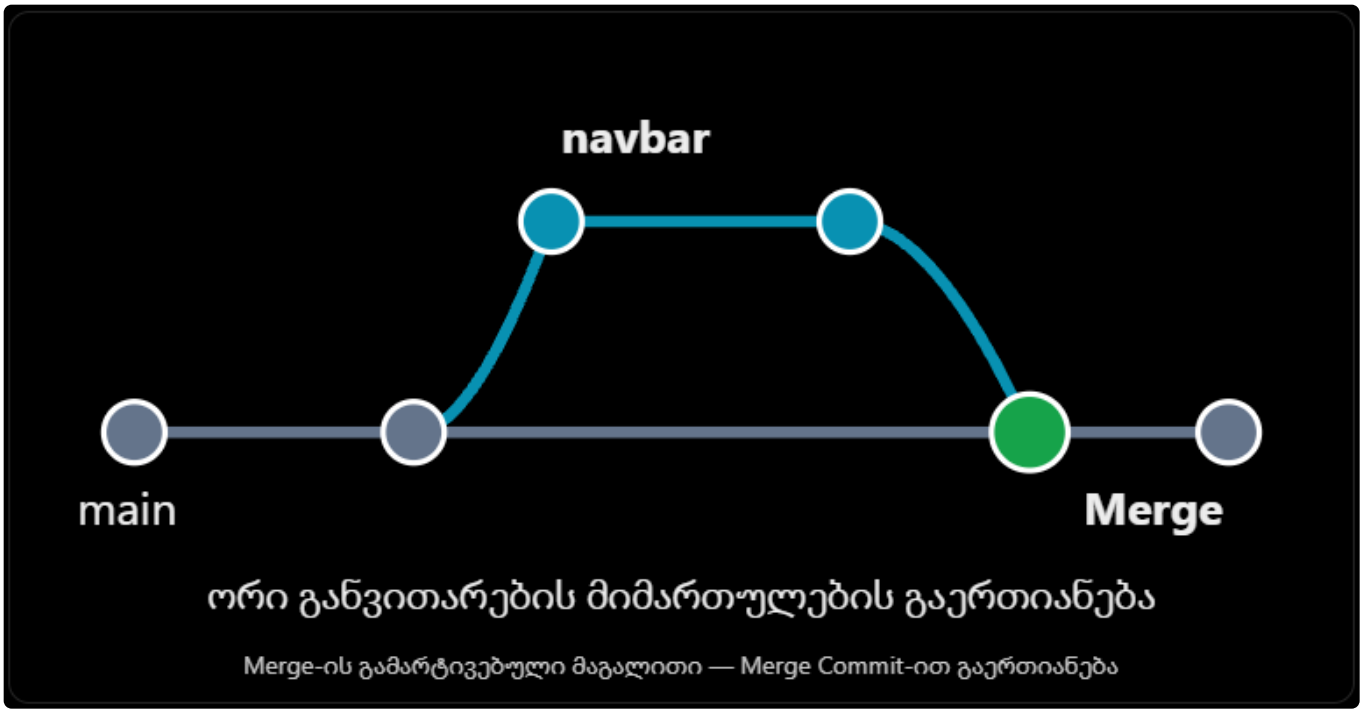
მაგალითად:

```
git checkout main
git merge navbar
```

პირველი ბრძანებით გადავდივართ `main` Branch-ზე.

მეორე ბრძანებით `navbar` Branch-ის ცვლილებებს ვაერთიანებთ `main`-ში.

ეს ნიშნავს, რომ ახლა მთავარი ვერსიაც შეიცავს ნავიგაციის მენიუს.



Git-ს გაერთიანების ორი გავრცელებული შემთხვევა აქვს:

Fast-forward Merge: თუ `main` Branch-ში ახალი commit არ შექმნილა მას შემდეგ, რაც ახალი Branch გამოვყავით, Git-ს შეუძლია `main` პირდაპირ წინ გადაიტანოს შესაბამის commit-ზე.

Three-way Merge: თუ ორივე Branch დამოუკიდებლად განვითარდა, Git ჩვეულებრივ ქმნის ახალ Merge Commit-ს, რომელიც მათ ისტორიებს აერთიანებს.

ორივე შემთხვევაში საბოლოო მიზანი ერთია — ცვლილებების გაერთიანება.

2.6. Merge-ის შესრულების მაგალითი

ვივარაუდოთ, რომ გვაქვს:

```
main
navbar
```

`navbar` Branch-ში უკვე დავამატეთ ნავიგაციის მენიუ და შევინახეთ commit.

ახლა უნდა გავაერთიანოთ.

ნაბიჯი 1 — გადადით მთავარ Branch-ზე

```
git checkout main
```

ნაბიჯი 2 — შეამოწმეთ პროექტის მდგომარეობა

```
git status
```

სასურველია, სამუშაო საქაღალდე სუფთა იყოს — შეუნახავი ცვლილებების გარეშე.

ნაბიჯი 3 — გააერთიანეთ Branch-ები

```
git merge navbar
```

თუ კონფლიქტი არ წარმოიშვა, Git წარმატებით გააერთიანებს ცვლილებებს.

ნაბიჯი 4 — შეამოწმეთ ისტორია

```
git log --oneline --graph
```

ნაბიჯი 5 — განახლებული main ატვირთეთ GitHub-ზე

```
git push origin main
```

ამის შემდეგ GitHub-ზე `main` Branch-შიც გამოჩნდება ახალი ცვლილებები.

2.7. რა არის Merge Conflict?

ზოგჯერ ორი Branch-ის გაერთიანების დროს Git ვერ წყვეტს, რომელი ცვლილება უნდა შეინარჩუნოს.

ამ მოვლენას ეწოდება Merge Conflict.

მაგალითად, თავდაპირველად `index.html` ფაილში წერია:

```
<h1>Welcome to Our Website</h1>
```

`main` Branch-ში ეს სათაური შევცვალეთ:

```
<h1>Welcome to Khoni School</h1>
```

ხოლო `design` Branch-ში იმავე სტრიქონზე სხვა ტექსტი ჩავწერეთ:

```
<h1>Welcome to Our New Website</h1>
```

ახლა `main` Branch-იდან ვასრულებთ:

```
git merge design
```

Git-მა შეიძლება ვერ გადაწყვიტოს, რომელი სათაური დატოვოს და გვაჩვენოს კონფლიქტი.

კონფლიქტის შემთხვევაში ფაილში შეიძლება გამოჩნდეს:

```
<<<<<<< HEAD
<h1>Welcome to Khoni School</h1>
=======
<h1>Welcome to Our New Website</h1>
>>>>>>> design
```

რას ნიშნავს ეს ნიშნები?

- `<<<<<<< HEAD` — მიმდინარე Branch-ის ცვლილების დასაწყისი;
- `=======` — ორი ვერსიის გამყოფი;
- `>>>>>>> design` — მეორე Branch-ის ცვლილების დასასრული.

2.8. როგორ მოვაგვაროთ Merge Conflict?

მოსწავლემ უნდა გადაწყვიტოს, რომელი ტექსტი სურს შეინარჩუნოს.

შეიძლება აირჩიოს პირველი ვარიანტი:

```
<h1>Welcome to Khoni School</h1>
```

მეორე ვარიანტი:

```
<h1>Welcome to Our New Website</h1>
```

ან შექმნას ახალი, გაერთიანებული ვარიანტი:

```
<h1>Welcome to Our New Khoni School Website</h1>
```

აუცილებელია კონფლიქტის ნიშნების წაშლა:

```
<<<<<<<
=======
>>>>>>>
```

შემდეგ ვინახავთ ფაილს.

```
git add index.html
git commit -m "Resolved merge conflict"
```

ამით Merge-ის პროცესი დასრულდება.

VS Code-ში კონფლიქტის დროს ხშირად გამოჩნდება შესაბამისი არჩევანის ღილაკებიც, მაგალითად:

- Accept Current Change;
- Accept Incoming Change;
- Accept Both Changes.

შეგვიძლია მათი გამოყენებაც, თუმცა მიღებული კოდი მაინც უნდა შევამოწმოთ.

მნიშვნელოვანია: Merge Conflict არ ნიშნავს, რომ Git გაფუჭდა ან პროექტი წაიშალა. ის ნიშნავს, რომ Git-ს ჩვენი გადაწყვეტილება სჭირდება.

3. პრაქტიკული მაგალითი — ორი Branch-ის გაერთიანება

შევქმნათ მარტივი ვებსაიტი, რომელსაც ეტაპობრივად დავამატებთ Header-სა და Footer-ს.

ნაბიჯი 1 — პროექტის მომზადება

გამოვიყენოთ GitHub-თან დაკავშირებული სასწავლო პროექტი.

`index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>School Website</title>
</head>
<body>

    <h1>Our School</h1>
    <p>Welcome to our website!</p>

</body>
</html>
```

შევამოწმოთ, რომ ვართ `main` Branch-ში და ყველა ცვლილება commit-შია შენახული.

```
git checkout main
git status
```

ნაბიჯი 2 — Header Branch-ის შექმნა

```
git checkout -b header
```

`index.html`-ში სათაურის ზემოთ დაამატეთ:

```
<header>
    <nav>
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Contact</a>
    </nav>
</header>
```

შეინახეთ ცვლილებები:

```
git add .
git commit -m "Added header"
```

ატვირთეთ GitHub-ზე:

```
git push -u origin header
```

ნაბიჯი 3 — Header-ის გაერთიანება Main Branch-ში

```
git checkout main
git merge header
```

ახლა `main` Branch-შიც გამოჩნდება Header.

ნაბიჯი 4 — Footer Branch-ის შექმნა

დარჩით განახლებულ `main` Branch-ზე და შექმენით:

```
git checkout -b footer
```

`index.html`-ში `</body>`-მდე დაამატეთ:

```
<footer>
    <p>© 2026 Khoni School</p>
    <p>All rights reserved.</p>
</footer>
```

შეინახეთ ცვლილებები:

```
git add .
git commit -m "Added footer"
```

ატვირთეთ:

```
git push -u origin footer
```

ნაბიჯი 5 — Footer-ის გაერთიანება

```
git checkout main
git merge footer
```

ახლა `main` Branch შეიცავს როგორც Header-ს, ისე Footer-ს.

ბოლოს:

```
git push origin main
```

გახსენით GitHub და შეამოწმეთ, რომ ორივე ცვლილება ატვირთულია.

4. პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — Branch-ის GitHub-ზე ატვირთვა (მარტივი)

დავალება: შექმენით ახალი Branch, დაამატეთ HTML სექცია და ატვირთეთ GitHub-ზე.

მოთხოვნები:

1. გადადით `main` Branch-ზე.
2. შექმენით ახალი Branch სახელით `subjects`.
3. `index.html` ფაილში დაამატეთ სექცია სათაურით `Our Subjects`.
4. სექციაში შექმენით სია მინიმუმ 5 სასწავლო საგნით.
5. ცვლილებები შეინახეთ commit-ით `Added subjects section`.
6. ატვირთეთ `subjects` Branch GitHub-ზე.

7. გახსენით GitHub და შეამოწმეთ, რომ Branch და მისი ცვლილებები გამოჩნდა.

მოსალოდნელი შედეგი: GitHub-ზე უნდა არსებობდეს `subjects` Branch, რომელშიც დამატებულია საგნების სექცია.

სავარჯიშო 2 — ორი Branch-ის გაერთიანება (საშუალო)

დავალება: შექმენით ორი Branch და მათი ცვლილებები გააერთიანეთ `main` Branch-ში.

მოთხოვნები:

1. `main` Branch-იდან შექმენით `about-section` Branch.
2. მასში დაამატეთ სკოლის შესახებ სექცია, სათაურითა და ორი პარაგრაფით.
3. შეინახეთ ცვლილებები commit-ში.
4. დაბრუნდით `main` Branch-ზე.
5. შექმენით `contact-section` Branch.
6. მასში დაამატეთ საკონტაქტო სექცია, სადაც იქნება ელფოსტა და ტელეფონის ნომერი.
7. შეინახეთ ცვლილებები commit-ში.
8. ატვირთეთ ორივე Branch GitHub-ზე.
9. გააერთიანეთ ჯერ `about-section`, შემდეგ `contact-section` Branch `main`-ში.
10. შეამოწმეთ, რომ საბოლოო ვებსაიტში ორივე სექცია არსებობს.
11. ატვირთეთ განახლებული `main` GitHub-ზე.

მოსალოდნელი შედეგი: `main` Branch-ში უნდა არსებობდეს სკოლის შესახებ და საკონტაქტო სექციები.

სავარჯიშო 3 — Merge Conflict-ის შექმნა და გამოსწორება (რთული)

დავალება: განზრახ შექმენით Merge Conflict და დამოუკიდებლად მოაგვარეთ.

გამოიყენეთ ისეთი სასწავლო რეპოზიტორია, რომელშიც ყველა წინა ცვლილება უკვე commit-შია შენახული.

მოთხოვნები:

1. გადადით `main` Branch-ზე.
2. `index.html` ფაილში მოძებნეთ მთავარი `h1` სათაური. თუ არ არსებობს, დაამატეთ და შეინახეთ commit-ში.
3. შექმენით ახალი Branch სახელით `new-title`.
4. ახალ Branch-ში შეცვალეთ მთავარი სათაური ტექსტით `Welcome to Our New School`.
5. შეინახეთ ცვლილებები commit-ში.
6. დაბრუნდით `main` Branch-ზე.

7. იმავე `h1` სათაურში ჩაწერეთ `Welcome to Khoni School`.
8. შეინახეთ ცვლილებები ცალკე commit-ში.
9. სცადეთ `new-title` Branch-ის გაერთიანება `main` Branch-ში.
10. დააკვირდით Merge Conflict-ის შეტყობინებას.
11. გახსენით კონფლიქტური ფაილი და დატოვეთ სათაური `Welcome to Khoni School Website`.
12. წაშალეთ კონფლიქტის ყველა სპეციალური ნიშანი.
13. შეინახეთ ფაილი.
14. გამოიყენეთ `git add` და `git commit` Merge-ის დასასრულებლად.
15. შეამოწმეთ Git-ის ისტორია და პროექტის მდგომარეობა.

მოსალოდნელი შედეგი: კონფლიქტი წარმატებით უნდა იყოს მოგვარებული და პროექტი კვლავ გამართულად მუშაობდეს.